

Neural Networks and Function Approximation

Multi-Layer Perceptrons in Scientific Machine Learning

Krishna Kumar

University of Texas at Austin

krishnak@utexas.edu

Outline

- 1 Part I: Motivation and Problem Setup
- 2 Part II: Universal Approximation
- 3 Part III: Building Neural Networks
- 4 Part IV: Training Neural Networks
- 5 Part V: Architecture Design
- 6 Part VI: Practical Considerations and Summary

The Central Challenge

Core Question

Can we build a machine that learns to approximate any continuous function from data?

Applications in Scientific Machine Learning:

- Solving partial differential equations (PDEs) without meshes
- Learning constitutive models from experimental data
- Discovering governing equations from observations
- Building surrogate models for optimization
- Uncertainty quantification in complex systems

What we'll explore:

- Mathematical foundations (Weierstrass $\rightarrow$ Universal Approximation)
- Building blocks (perceptron $\rightarrow$ deep networks)
- Training algorithms (gradient descent $\rightarrow$ backpropagation)
- Practical considerations (architecture, regularization)

The Benchmark Problem: 1D Poisson Equation

Problem formulation:

$$\begin{cases} -\nabla^2 u = f & \text{in } \Omega \\ u = g & \text{on } \partial\Omega \end{cases} \quad (1)$$

1D case:

$$-\frac{d^2 u}{dx^2} = f(x), \quad u(0) = u(1) = 0$$

For the source term $f(x) = \pi^2 \sin(\pi x)$

Analytical solution: $u(x) = \sin(\pi x)$

Challenge: Can we learn this solution from sparse, noisy measurements?

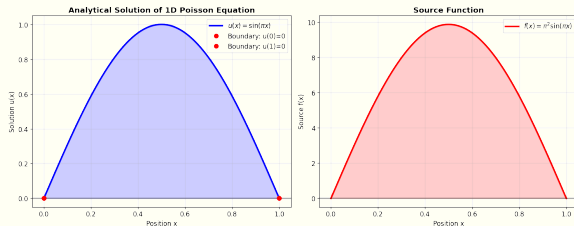


Fig. Analytical solution showing parabolic profile

The Function Approximation Challenge

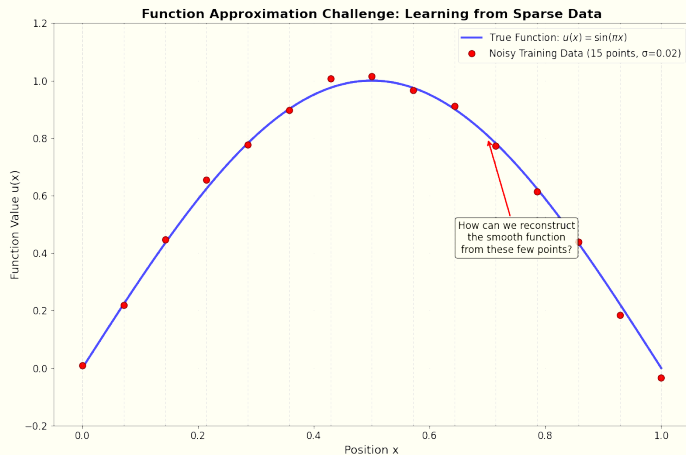


Fig. Learning from sparse data: 15 noisy measurements to reconstruct continuous solution

The Function Approximation Challenge

Given:

- Sparse measurements (e.g., 15 points)
- Noisy observations
- Unknown functional form
- Potentially high dimensions

Goal:

- Reconstruct continuous solution
- Quantify uncertainty
- Generalize to new domains
- Handle complex geometries

Traditional Methods: Finite Differences

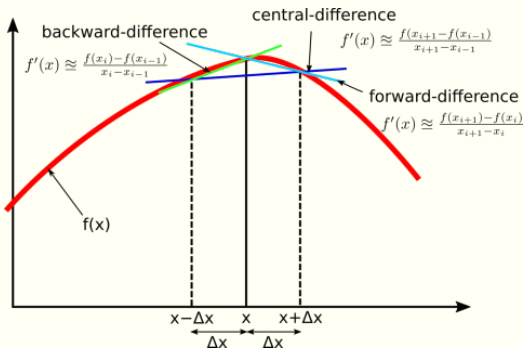


Fig. Finite difference discretization

The approach:

- Discretize domain into grid
- Approximate derivatives:
 $\frac{d^2 u}{dx^2} \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2}$
- Solve linear system: $Au = b$

Limitations:

- Fixed grid resolution
- Curse of dimensionality ($O(N^d)$)
- Poor for irregular domains
- Cannot learn from data
- No continuous representation

Finite Difference Results

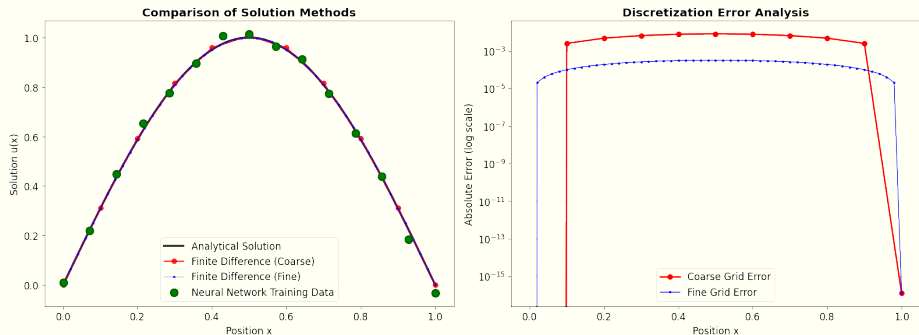


Fig. Finite difference solution: accurate but discrete, requires mesh

Key observation: We need a flexible, continuous, learnable representation

From Finitely Many Numbers to a Network

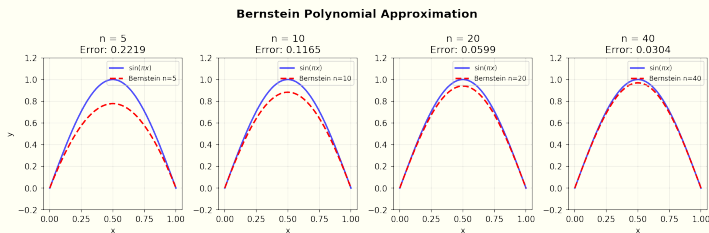
The representation lecture stored a function as a short list of numbers and a rule that rebuilds the curve, readings at stations joined by straight lines, or coefficients on a few fixed shapes. Each choice approximates every function outside its own family, and the error is measured in a norm the physics chooses. Weierstrass's theorem says polynomials reach every continuous function, and Bernstein's coin flips build the polynomial, with no word about the degree, the weights, or the slopes.

A hat function is three ReLU ramps, so the straight-segment reconstruction is already a one-hidden-layer ReLU network with its kinks at the stations. This lecture lets the kinks move.

The Weierstrass approximation theorem

Theorem (Weierstrass, 1885)

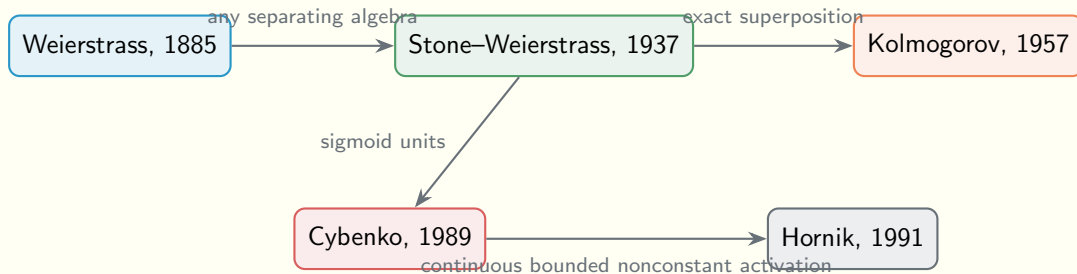
For every continuous f on $[a, b]$ and every $\epsilon > 0$ there is a polynomial p with $|f(x) - p(x)| < \epsilon$ for every x in $[a, b]$.



Bernstein $B_n f$ for $\sin(\pi x)$,
largest gap
0.222, 0.117, 0.060,
0.030 at
 $n = 5, 10, 20, 40$

what the theorem does not
say
**the degree, the
weights, or the
slopes**

History of universal approximation theorems



what these theorems give
**existence of a
representation or
approximation**

what none of them gives
**a procedure that
trains it**

Universal Approximation Theorem

Theorem (Cybenko, 1989)

Let σ be a continuous, bounded, non-constant, monotonically increasing function. Then finite sums of the form:

$$G(x) = \sum_{i=1}^N \alpha_i \sigma(w_i^T x + b_i)$$

are dense in $C([0, 1]^n)$ under the supremum norm.

In plain English: A single hidden layer neural network can approximate any continuous function to arbitrary accuracy, given enough neurons.

Key points:

- Existence guarantee, not constructive
- Says nothing about number of neurons needed
- Doesn't tell us how to find the weights

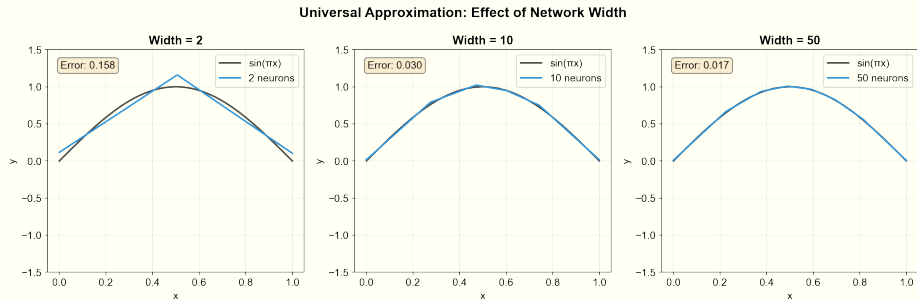


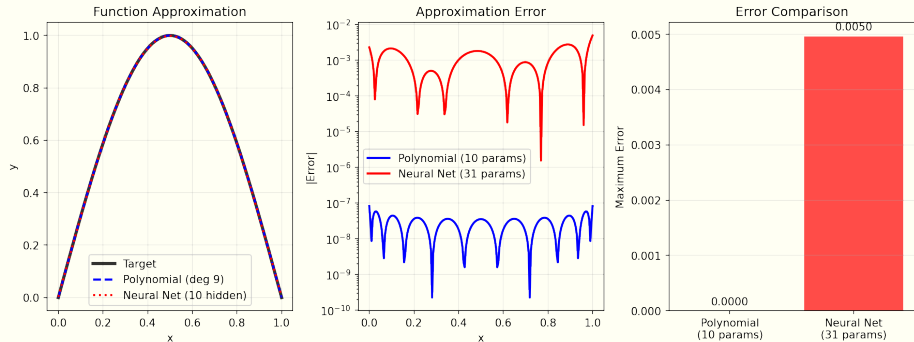
Fig. Neural network approximation with increasing width: 2, 10, and 50 neurons

Observations:

- More neurons $\rightarrow$ better approximation
- Each neuron adds a "bump" or feature
- Adaptive placement of features
- No fixed polynomial structure

Polynomial and neural network approximation of $\sin(\pi x)$

Weierstrass (Polynomial) vs Universal Approximation (Neural Network)



polynomial, ten numbers
largest gap below
 10^{-7}

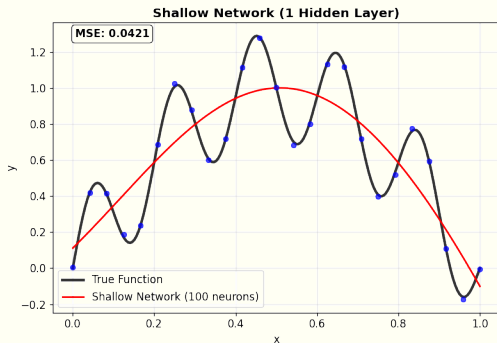
sigmoid network, thirty-one
numbers, one training run
largest gap 0.0050

Polynomials and neural networks compared

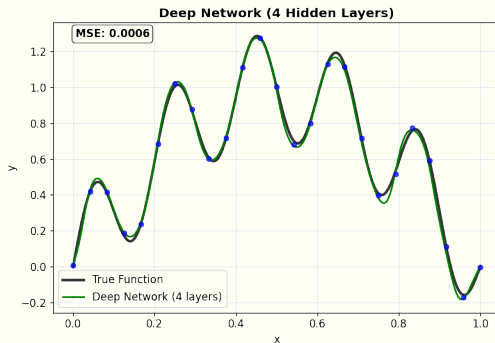
	polynomials	one-hidden-layer ReLU networks
shapes	global, every coefficient changes the curve everywhere	ramps, each unit changes the curve beyond its breakpoint
placement	fixed by the degree	breakpoints are trained parameters
failure on sparse data	Runge oscillations near the ends	a descent that stalls
growth with dimension	$\binom{n+d}{d}$ coefficients at degree n in d variables	$m(d+2) + 1$ parameters for m units, and the m needed for a given accuracy can still grow fast with d
structure	one sum	layers compose

Shallow and deep networks on a high-frequency function

High-Frequency Function: Shallow vs Deep



one hidden layer, a
hundred units
mean squared error
0.0421



four hidden layers
mean squared error
0.0006

The Perceptron: Building Block of Neural Networks

Mathematical model:

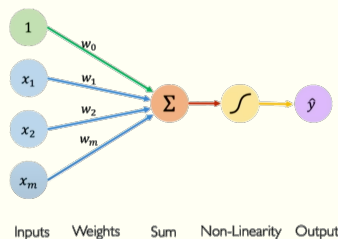
$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b) \quad (2)$$

where:

- $\mathbf{x} \in \mathbb{R}^n$: input features
- $\mathbf{w} \in \mathbb{R}^n$: weights
- $b \in \mathbb{R}$: bias
- $\sigma(\cdot)$: activation function
- $\hat{y}$: output/prediction

Geometric interpretation:

- Linear decision boundary
- Hyperplane: $\mathbf{w}^T \mathbf{x} + b = 0$
- Normal vector: $\mathbf{w}$



The diagram shows the mathematical formula for the perceptron model: $\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$. Annotations include: a purple arrow pointing to $\hat{y}$ labeled 'Output'; a red arrow pointing to the sum term $w_0 + \sum_{i=1}^m x_i w_i$ labeled 'Linear combination of inputs'; a yellow arrow pointing to the function g labeled 'Non-linear activation function'; and a green arrow pointing to w_0 labeled 'Bias'.

Fig. Perceptron model: linear combination followed by activation

Linear Perceptron Implementation

```
import numpy as np

class LinearPerceptron:
    def __init__(self, input_dim):
        # Initialize weights randomly
        self.w = np.random.randn(input_dim) * 0.01
        self.b = np.random.randn() * 0.01

    def forward(self, x):
        """Compute output for input x"""
        return np.dot(x, self.w) + self.b

    def sigmoid(self, z):
        """Sigmoid activation function"""
        return 1 / (1 + np.exp(-z))

    def predict(self, x):
        """Make binary prediction"""
        z = self.forward(x)
        return self.sigmoid(z) > 0.5

    def train(self, X, y, lr=0.01, epochs=100):
        """Train using gradient descent"""
        for epoch in range(epochs):
            # Forward pass
            z = self.forward(X)
            y_pred = self.sigmoid(z)

            # Compute loss (binary cross-entropy)
            loss = -np.mean(y * np.log(y_pred + 1e-8) +
```

Training a Linear Model

Gradient Descent:

$$\mathcal{L} = \frac{1}{2} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (3)$$

$$\frac{\partial \mathcal{L}}{\partial w_j} = - \sum_{i=1}^n (y_i - \hat{y}_i) x_{ij} \quad (4)$$

$$w_j \leftarrow w_j - \eta \frac{\partial \mathcal{L}}{\partial w_j} \quad (5)$$

Learning process:

- Start with random weights
- Compute predictions
- Calculate error
- Update weights in direction of negative gradient

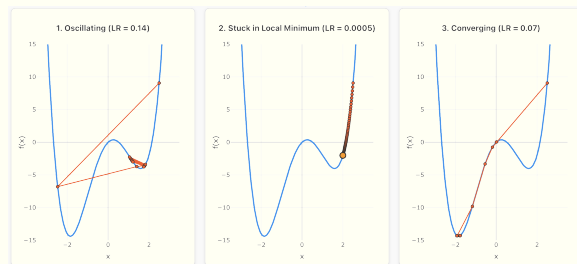


Fig. Gradient descent trajectory in weight space

Linear Models Work for Linearly Separable Data

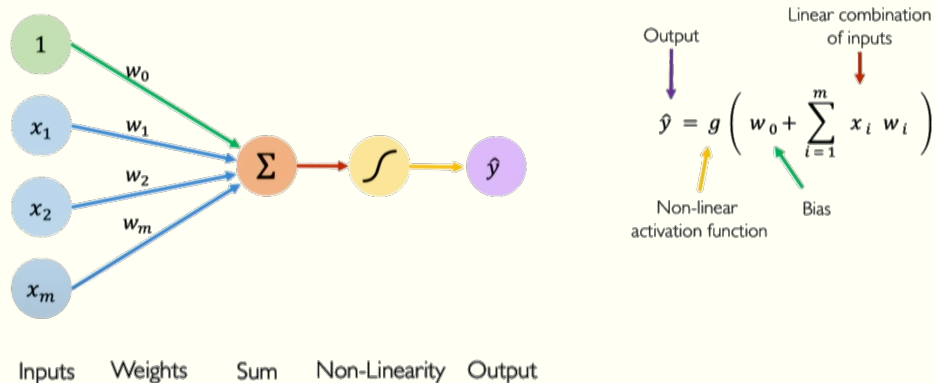


Fig. Linear perceptron successfully classifies linearly separable data

Success cases: Simple classification problems and Linear regression

The Critical Limitation: Linear Models Fail

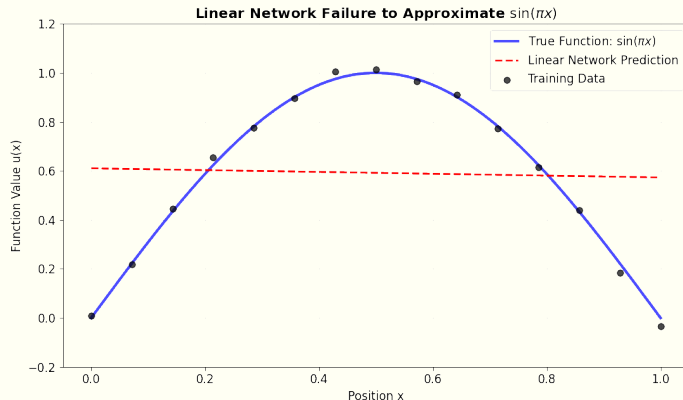


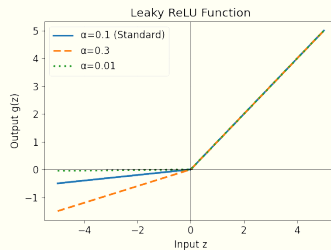
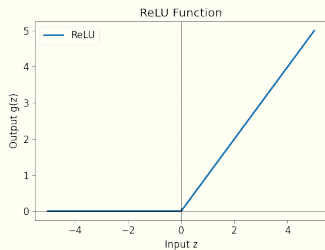
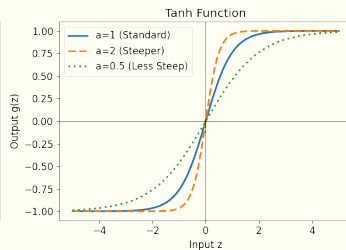
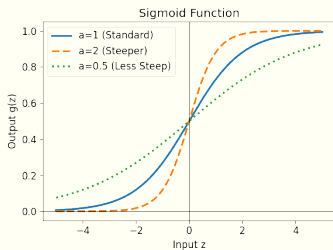
Fig. Linear models cannot approximate nonlinear functions like $\sin(\pi x)$

Fundamental problem: Composition of linear functions is still linear!

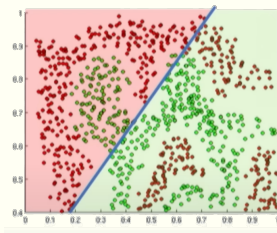
$$f(x) = W_2(W_1(W_0 x)) = (W_2 W_1 W_0)x = Wx$$

The Solution: Activation Functions

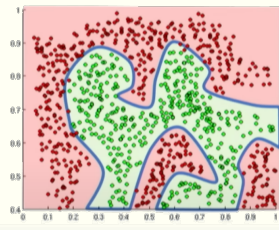
Common Activation Functions (Parameterized)



Why Nonlinearity Matters



Linear activation functions produce linear decisions no matter the network size



Non-linearities allow us to approximate arbitrarily complex functions

Fig. Without activation functions, deep networks collapse to linear models

$$\text{Without } \sigma : \quad h_1 = W_1 x + b_1 \quad (6)$$

$$h_2 = W_2 h_1 + b_2 = W_2 (W_1 x + b_1) + b_2 \quad (7)$$

$$= (W_2 W_1) x + (W_2 b_1 + b_2) = W x + b \quad (8)$$

Single Hidden Layer Networks

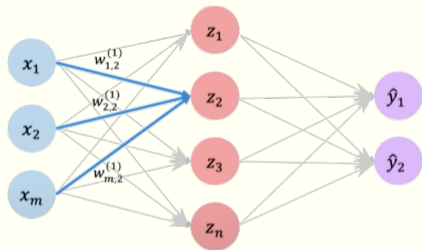


Fig. Single hidden layer network with multiple neurons

Architecture:

$$h_i = \sigma(w_i^T x + b_i) \quad i = 1, \dots, M$$

$$y = \sum_{i=1}^M v_i h_i + c$$

Capabilities:

- Universal approximation with enough neurons
- Each neuron adds a "feature" or "basis function"
- Output is weighted combination of features

Function Spaces: The Mathematical Setting

Banach Spaces:

- Complete normed vector spaces
- $C([0, 1])$ with $\|f\|_\infty = \max_x |f(x)|$
- $L^p([0, 1])$ with $\|f\|_p = (\int |f|^p)^{1/p}$

Hilbert Spaces:

- Inner product spaces
- $L^2([0, 1])$ with $\langle f, g \rangle = \int fg$
- Orthogonal projections
- Energy minimization

Density Results:

A set S is *dense* in X if:

$$\forall f \in X, \forall \epsilon > 0, \exists g \in S : \|f - g\| < \epsilon$$

Neural networks are dense in:

- $C([0, 1])$ - continuous functions
- $L^p([0, 1])$ - integrable functions
- $H^1([0, 1])$ - Sobolev spaces

Implication: Can approximate solutions to PDEs!

Constructive Proof: Building Functions with ReLU

ReLU (Rectified Linear Unit):

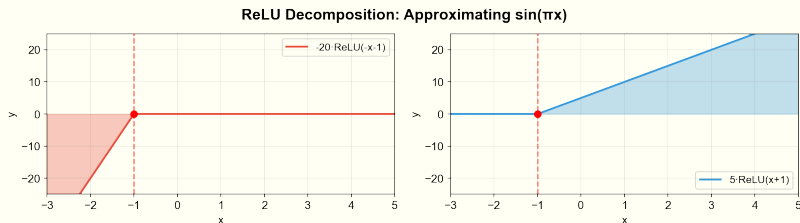
$$\text{ReLU}(x) = \max(0, x)$$

Building blocks:

$$\text{Bump function: } \text{ReLU}(x - a) - \text{ReLU}(x - b) \quad (11)$$

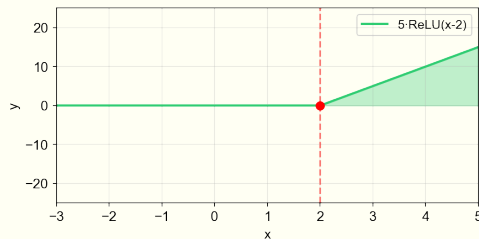
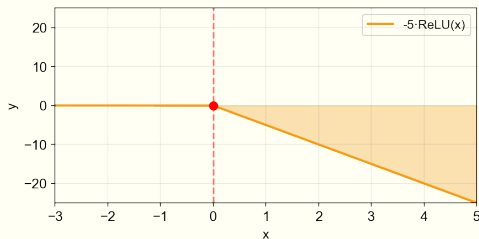
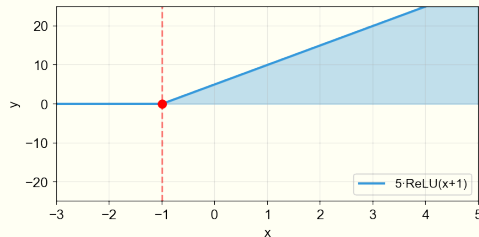
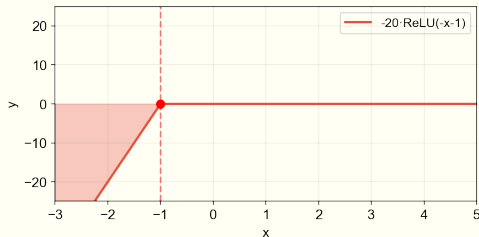
$$\text{Triangle: } \text{ReLU}(x) - 2 \text{ReLU}(x - 1) + \text{ReLU}(x - 2) \quad (12)$$

$$\text{Step approx: } \frac{1}{h} [\text{ReLU}(x) - \text{ReLU}(x - h)] \quad (13)$$



ReLU Networks: Piecewise Linear Approximation

ReLU Decomposition: Approximating $\sin(\pi x)$



Final Approximation

The Role of Bias: Controlling Breakpoints

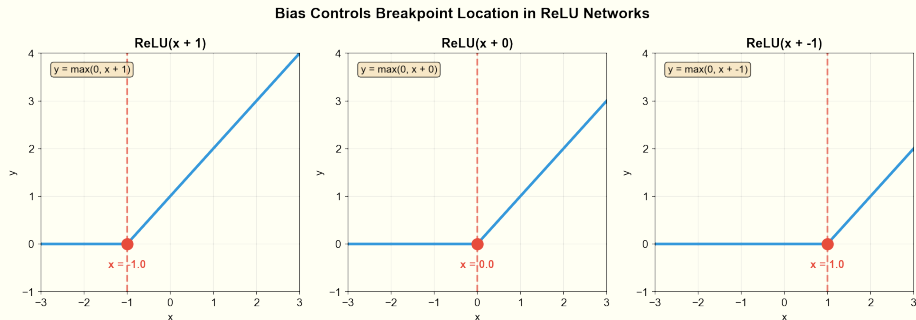


Fig. Bias parameters control where ReLU units activate

For neuron $h = \text{ReLU}(wx + b)$:

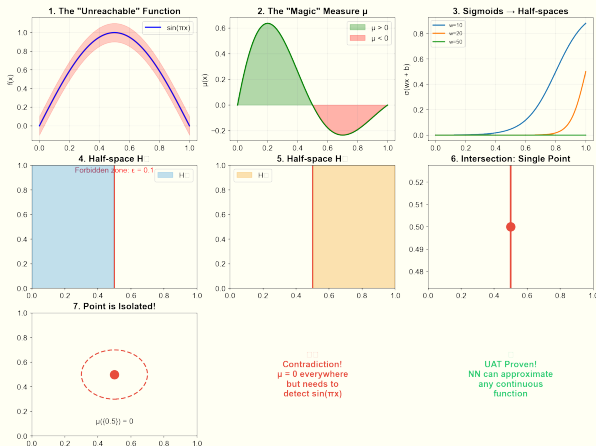
- Activates when $w x + b = 0$
- Breakpoint at $x = -b/w$
- Bias shifts the activation threshold

Proof Strategy: Hahn-Banach Argument

Strategy:

- 1 Assume neural networks are NOT dense
- 2 Apply Hahn-Banach theorem
- 3 Find separating functional
- 4 Show functional must be zero
- 5 Reach contradiction

Proof by Contradiction: Universal Approximation Theorem



The Continuity Requirement

What UAT guarantees:

- Continuous functions ✓
- Smooth functions ✓
- Piecewise continuous ✓ (away from jumps)

What UAT doesn't guarantee:

- Discontinuous functions ✗
- Delta functions ✗
- Step functions ✗ (pointwise)

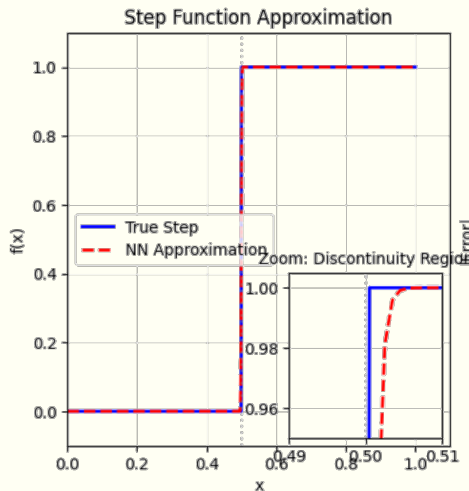


Fig. Neural networks approximate discontinuities in L^2 , not pointwise

Which Activations Work? – Requirements for UAT

- Non-polynomial
- Non-constant
- Bounded (for some theorems)
- Continuous (for some theorems)

Common choices:

Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$

- Smooth, bounded
- Vanishing gradients
- Historical choice

Tanh: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

- Zero-centered
- Bounded $[-1, 1]$
- Still saturates

ReLU: $\text{ReLU}(x) = \max(0, x)$

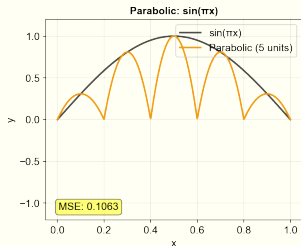
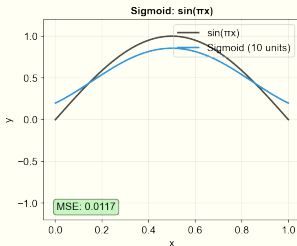
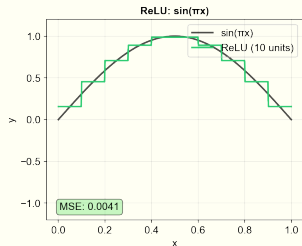
- Simple, efficient
- No saturation
- Dead neurons problem

Modern variants:

- Leaky ReLU: $\max(0.01x, x)$
- ELU, SELU, Swish
- GELU (Transformers)

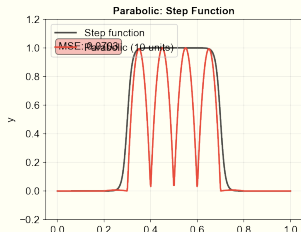
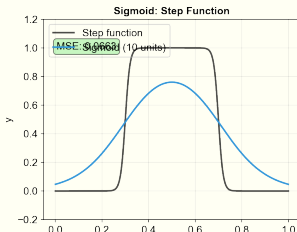
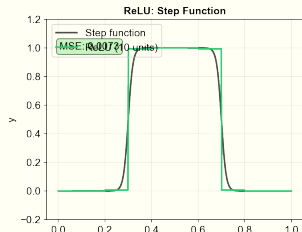
Parabolic Activations Don't Work

Activation Function Comparison: ReLU & Sigmoid (UAT) vs Parabolic (Non-UAT)



Parabolic activation ($4t(1-t)$ bumps) may work for specific smooth functions like $\sin(\pi x)$ but fails for discontinuous or sharp transitions. ReLU and Sigmoid are universal approximators.

Works for this specific function



The XOR Problem: Historical Crisis

XOR Truth Table:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

The problem:

- No single line can separate the classes
- Points (0,1) and (1,0): class 1
- Points (0,0) and (1,1): class 0
- Diagonally opposite points same class

Historical impact: Minsky & Papert (1969) → First AI Winter

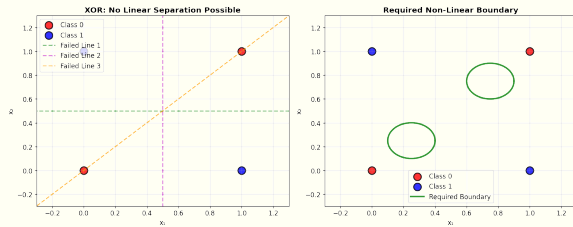


Fig. XOR problem: not linearly separable

XOR Implementation: Failure of Linear Model

```
import numpy as np
import matplotlib.pyplot as plt

# XOR dataset
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 1, 1, 0])

# Train linear perceptron
perceptron = LinearPerceptron(input_dim=2)
perceptron.train(X, y, lr=0.1, epochs=1000)

# Results after training:
# The linear model achieves at best 50% accuracy
# It can only correctly classify 2 out of 4 points

# Visualization shows no linear boundary exists
# that can separate XOR classes
```

Result: Linear model fails completely on XOR

- Best accuracy: 50% (random guessing!)
- Cannot learn the pattern
- Need nonlinearity

Multi-Layer Networks: Solving XOR

Architecture:

- Input layer: 2 neurons (x_1, x_2)
- Hidden layer: 2 neurons
- Output layer: 1 neuron
- Activation: Sigmoid/ReLU

How it works:

- Hidden neuron 1: (x_1 OR x_2)
- Hidden neuron 2: (x_1 AND x_2)
- Output: h_1 AND NOT h_2

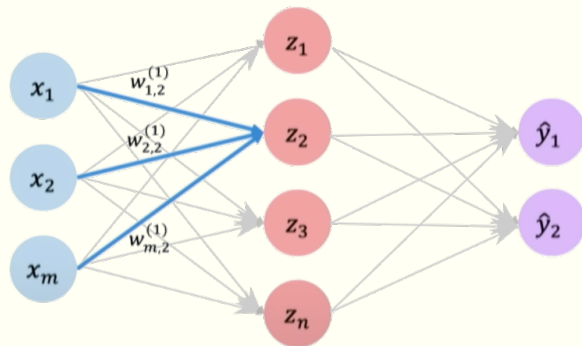


Fig. Two-layer network architecture for XOR

XOR Solution: Decision Boundaries

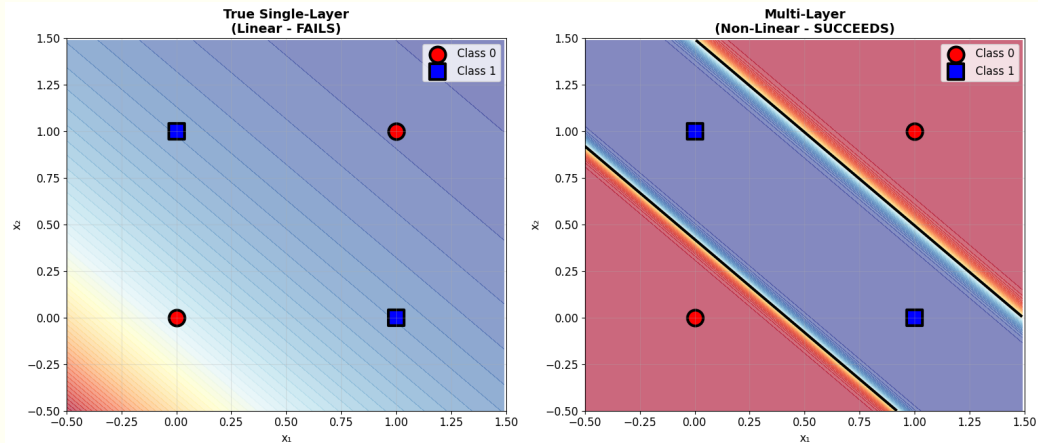


Fig. Hidden layer creates linearly separable feature space

Key insight: Hidden layer transforms the input space!

XOR Solution: Implementation

```
class TwoLayerNetwork:
    def __init__(self, input_dim, hidden_dim, output_dim):
        # Initialize weights
        self.W1 = np.random.randn(input_dim, hidden_dim) * 0.5
        self.b1 = np.zeros(hidden_dim)
        self.W2 = np.random.randn(hidden_dim, output_dim) * 0.5
        self.b2 = np.zeros(output_dim)

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def forward(self, X):
        # Hidden layer
        self.z1 = X @ self.W1 + self.b1
        self.a1 = self.sigmoid(self.z1)

        # Output layer
        self.z2 = self.a1 @ self.W2 + self.b2
        self.a2 = self.sigmoid(self.z2)

        return self.a2

    def backward(self, X, y, lr=0.1):
        m = len(y)

        # Output layer gradients
        dz2 = self.a2 - y.reshape(-1, 1)
        dW2 = self.a1.T @ dz2 / m
        db2 = np.sum(dz2, axis=0) / m
```

Multi-Output Networks

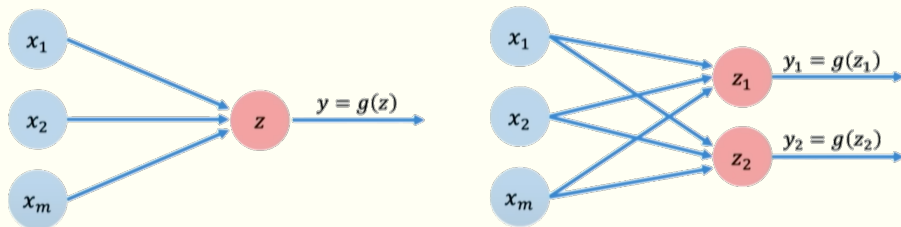


Fig. Networks can have multiple outputs for multi-task learning

Applications:

- Multi-class classification (softmax output)
- Multi-task learning
- Vector field prediction (PDEs)
- Uncertainty quantification (mean + variance)

The Training Process

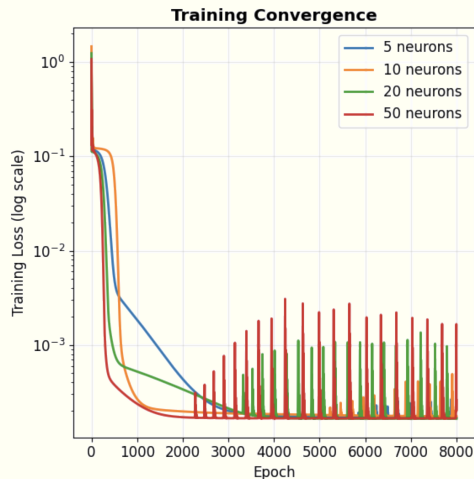


Fig. Training loss decreases over iterations

Training loop:

- 1 Initialize weights randomly
- 2 Forward pass: compute predictions
- 3 Compute loss: $\mathcal{L}(y, \hat{y})$
- 4 Backward pass: compute gradients
- 5 Update weights: $w \leftarrow w - \eta \nabla_w \mathcal{L}$
- 6 Repeat until convergence

Computing Gradients: Three Approaches

Symbolic:

- Manipulate expressions
- Exact derivatives
- Expression swell

Example:

$$\frac{d}{dx}[x^2 \sin(x)]$$

$$= 2x \sin(x) + x^2 \cos(x)$$

Numerical:

- Finite differences
- Approximation
- Round-off errors

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

$$\text{Error: } O(h) + O(1/h)$$

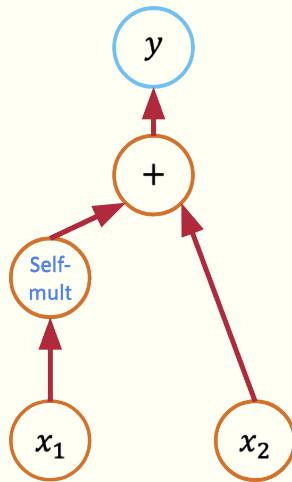
Automatic:

- Chain rule
- Machine precision
- Efficient

Decompose into elementary ops, apply chain rule

For neural networks: Automatic differentiation (backpropagation) is the only viable option

Automatic Differentiation: The Key Innovation

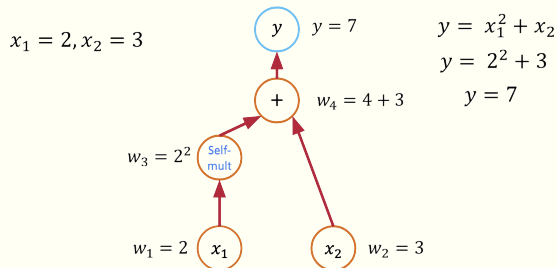


$$y = x_1^2 + x_2$$

Fig. Computational graph for automatic differentiation

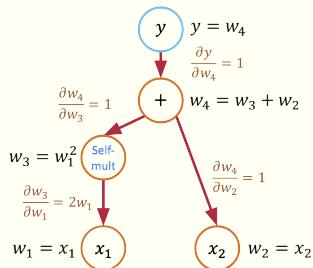
Forward Mode vs Reverse Mode AD

Forward Mode:



- Compute $\dot{y} = \frac{\partial y}{\partial x_i}$
- Propagate derivatives forward
- One pass per input dimension
- Good for: $\mathbb{R}^n \rightarrow \mathbb{R}^m, n \ll m$

Reverse Mode:



$$y = x_1^2 + x_2$$

$$\nabla y = \left(\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2} \right)$$

$$\nabla y = (2x_1, 1)$$

We can find the gradient with respect to the output y

Using $x_1 = 2, x_2 = 3$:

$$\nabla y = (4, 1)$$

- Compute $\bar{x} = \frac{\partial L}{\partial x}$
- Propagate gradients backward
- One pass for all gradients
- Good for: $\mathbb{R}^n \rightarrow \mathbb{R}, n \gg 1$

Backpropagation = Reverse Mode AD

Why reverse mode for neural networks?

- Loss is scalar: $\mathcal{L} : \mathbb{R}^n \rightarrow \mathbb{R}$
- Many parameters (thousands to billions)
- Need all gradients $\frac{\partial \mathcal{L}}{\partial w_i}$
- Reverse mode: ONE backward pass gets ALL gradients!

The algorithm:

- ➊ **Forward pass:** Compute and store all activations
- ➋ **Backward pass:** Starting from loss, propagate gradients

$$\text{Forward: } a^{(l)} = \sigma(W^{(l)} a^{(l-1)} + b^{(l)}) \quad (14)$$

$$\text{Backward: } \delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot \sigma'(z^{(l)}) \quad (15)$$

Backpropagation: Mathematical Details of Forward Pass

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

$$a^{(1)} = \sigma(z^{(1)})$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$

$$\hat{y} = \sigma(z^{(2)})$$

$$\mathcal{L} = \frac{1}{2} \|\hat{y} - y\|^2$$

Backpropagation: Mathematical Details of Backward Pass

$$\delta^{(2)} = (\hat{y} - y) \odot \sigma'(z^{(2)})$$

$$\nabla_{W^{(2)}} \mathcal{L} = \delta^{(2)} (a^{(1)})^T$$

$$\nabla_{b^{(2)}} \mathcal{L} = \delta^{(2)}$$

$$\delta^{(1)} = (W^{(2)})^T \delta^{(2)} \odot \sigma'(z^{(1)})$$

$$\nabla_{W^{(1)}} \mathcal{L} = \delta^{(1)} x^T$$

$$\nabla_{b^{(1)}} \mathcal{L} = \delta^{(1)}$$

Backpropagation Implementation

```
def backpropagation(X, y, W1, b1, W2, b2):  
    """  
    Compute gradients using backpropagation  
    """  
    m = X.shape[0]  
  
    # Forward pass  
    z1 = X @ W1 + b1  
    a1 = sigmoid(z1)  
    z2 = a1 @ W2 + b2  
    a2 = sigmoid(z2)  
  
    # Compute loss  
    loss = 0.5 * np.sum((a2 - y)**2) / m  
  
    # Backward pass  
    # Output layer  
    delta2 = (a2 - y) * sigmoid_derivative(z2)  
    dW2 = a1.T @ delta2 / m  
    db2 = np.sum(delta2, axis=0, keepdims=True) / m  
  
    # Hidden layer  
    delta1 = (delta2 @ W2.T) * sigmoid_derivative(z1)  
    dW1 = X.T @ delta1 / m  
    db1 = np.sum(delta1, axis=0, keepdims=True) / m  
  
    return dW1, db1, dW2, db2, loss  
  
def sigmoid_derivative(z):  
    s = sigmoid(z)
```

Gradient Descent Variants

Batch Gradient Descent:

- Use entire dataset
- Exact gradient
- Slow for large datasets

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t; \mathcal{D})$$

Stochastic GD (SGD):

- One sample at a time
- Noisy gradient
- Fast but unstable

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t; x_i)$$

Mini-batch GD:

- Best of both worlds
- Batch size: 32-512 typical

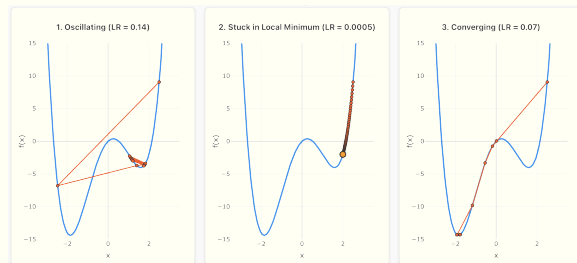


Fig. SGD trajectory: noisy but effective

Momentum:

$$v_{t+1} = \beta v_t + (1 - \beta)g_t \quad (16)$$

$$\theta_{t+1} = \theta_t - \eta v_{t+1} \quad (17)$$

Accelerates convergence, reduces oscillations

Adam (Adaptive Moment Estimation):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t \quad (1\text{st moment}) \quad (18)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2 \quad (2\text{nd moment}) \quad (19)$$

$$\hat{m}_t = m_t / (1 - \beta_1^t) \quad (\text{bias correction}) \quad (20)$$

$$\hat{v}_t = v_t / (1 - \beta_2^t) \quad (21)$$

$$\theta_t = \theta_{t-1} - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) \quad (22)$$

Benefits: Adaptive learning rates per parameter, works well in practice

Loss Functions for Different Problems

Regression:

- MSE: $\mathcal{L} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$
- MAE: $\mathcal{L} = \frac{1}{n} \sum |y_i - \hat{y}_i|$
- Huber: Robust to outliers

Classification:

- Cross-entropy:
 $\mathcal{L} = - \sum y_i \log(\hat{y}_i)$
- Hinge loss (SVM)
- Focal loss (imbalanced)

Physics-Informed:

- PDE residual:
 $\mathcal{L}_{PDE} = \|\mathcal{N}[u_\theta]\|^2$
- Boundary conditions:
 $\mathcal{L}_{BC} = \|u_\theta|_{\partial\Omega} - g\|^2$
- Initial conditions:
 $\mathcal{L}_{IC} = \|u_\theta|_{t=0} - u_0\|^2$

Total loss:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{data} + \lambda_2 \mathcal{L}_{PDE} + \lambda_3 \mathcal{L}_{BC}$$

Training Challenges: Overfitting

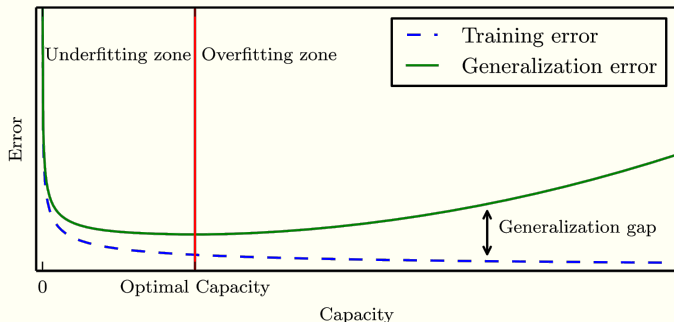


Fig. Overfitting: model memorizes training data, fails to generalize

Signs of overfitting:

- Training loss $\downarrow$, validation loss $\uparrow$
- Large gap between training and validation performance
- Model too complex for data

Regularization: Preventing Overfitting

The Problem

Models that are too complex memorize training data instead of learning patterns

Regularization adds a penalty term to the loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \cdot \mathcal{R}(\theta)$$

Classical methods:

- **L1 regularization:** Promotes sparsity (feature selection)
- **L2 regularization:** Prevents large weights (weight decay)
- **Early stopping:** Stop training before overfitting
- **Data augmentation:** Artificially expand training set

Modern methods:

- **Dropout:** Randomly disable neurons during training
- **Batch/Layer normalization:** Stabilize internal distributions
- **Mixup, Cutout:** Advanced data augmentation

L1 vs L2 Regularization: Geometric Intuition

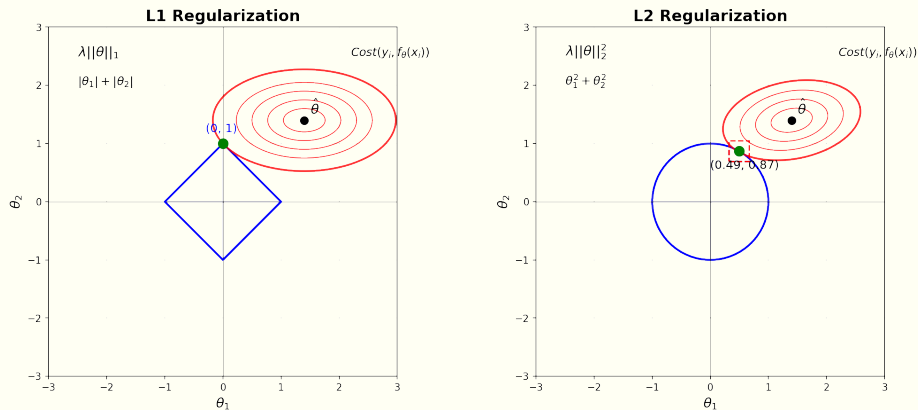


Fig. L1 (diamond) vs L2 (circle) constraint regions and their effect on optimal solutions

Key insight: The shape of the constraint region determines sparsity!

L1 Regularization (Lasso)

Mathematical form:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \sum_{i=1}^n |w_i|$$

Gradient:

$$\frac{\partial \mathcal{R}_{L1}}{\partial w_i} = \text{sign}(w_i)$$

Key properties:

- Creates sparse models (many weights = 0)
- Automatic feature selection
- Non-differentiable at zero
- Diamond-shaped constraint region

When to use L1:

- High-dimensional data
- Many irrelevant features
- Need interpretability
- Want feature selection

Effect on weights:

- Small weights $\rightarrow$ pushed to exactly 0
- Large weights $\rightarrow$ reduced by constant amount
- Result: sparse weight matrix

L2 Regularization (Ridge)

Mathematical form:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \sum_{i=1}^n w_i^2$$

Gradient:

$$\frac{\partial \mathcal{R}_{L2}}{\partial w_i} = 2w_i$$

Key properties:

- Shrinks all weights uniformly
- Weights approach but don't reach zero
- Smooth, differentiable everywhere
- Circular constraint region

When to use L2:

- Multicollinearity present
- All features potentially relevant
- Need stable solution
- Prevent weight explosion

Effect on weights:

- All weights decay proportionally
- Large weights penalized more
- Result: small but non-zero weights

Comparison: L1 vs L2 Regularization

Feature	L1 (Lasso)	L2 (Ridge)
Penalty term	$\sum w_i $	$\sum w_i^2$
Constraint shape	Diamond	Circle
Feature selection	Yes	No
Sparsity	High	Low
Solution uniqueness	May not be unique	Always unique
Computational cost	Higher (subgradient)	Lower (gradient)
Robustness to outliers	More robust	Less robust
Multicollinearity	Can be unstable	Handles well

Elastic Net: Combines both L1 and L2

$$\mathcal{R}_{\text{elastic}} = \alpha \sum |w_i| + (1 - \alpha) \sum w_i^2$$

Best of both worlds: feature selection + stability

Implementing Regularization in Neural Networks

```
1 import torch
2 import torch.nn as nn
3
4 class RegularizedNet(nn.Module):
5     def __init__(self, input_dim, hidden_dim, output_dim,
6                 reg_type='l2', lambda_reg=0.01):
7         super().__init__()
8         self.fc1 = nn.Linear(input_dim, hidden_dim)
9         self.fc2 = nn.Linear(hidden_dim, output_dim)
10        self.reg_type = reg_type
11        self.lambda_reg = lambda_reg
12
13    def regularization_loss(self):
14        reg_loss = 0
15        for param in self.parameters():
16            if self.reg_type == 'l1':
17                reg_loss += torch.sum(torch.abs(param))
18            elif self.reg_type == 'l2':
19                reg_loss += torch.sum(param ** 2)
20        return self.lambda_reg * reg_loss
21
22    def total_loss(self, data_loss):
23        return data_loss + self.regularization_loss()
```

Choosing the Regularization Parameter λ

Too small λ :

- Insufficient regularization
- Model still overfits
- High variance

Too large λ :

- Over-regularization
- Model underfits
- High bias

Finding optimal λ :

- Cross-validation
- Grid search: $\lambda \in [10^{-4}, 10^{-3}, \dots, 10^2]$
- Validation curves

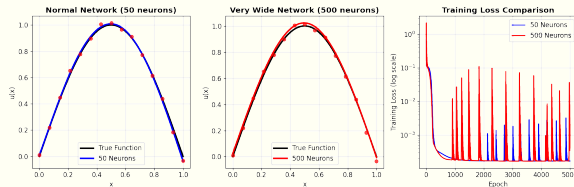
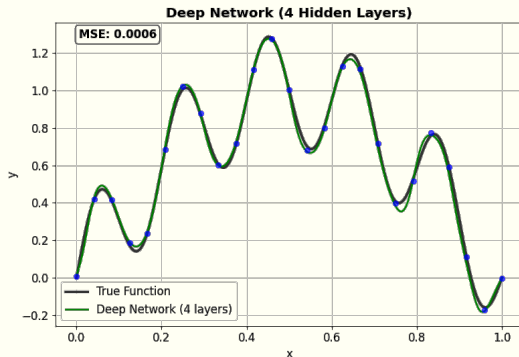
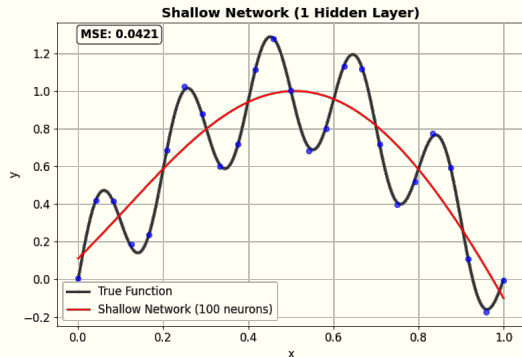


Fig. Effect of regularization strength on model complexity

Width vs Depth Trade-offs

High-Frequency Function: Shallow vs Deep



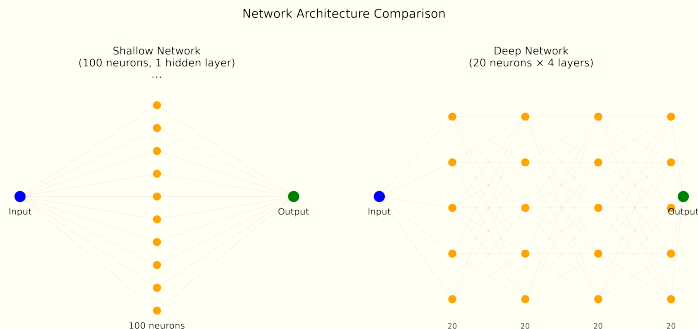
Shallow (Wide) Networks:

- 100 neurons, 1 hidden layer
- Universal approximator
- May need exponential width

Deep (Narrow) Networks:

- 20 neurons $\times$ 4 layers
- Compositional efficiency
- Better feature hierarchy

Network Architecture Comparison



- **Parameters:** Deep network has more total parameters but better efficiency
- **Shallow:** $100 \text{ input-to-hidden} + 100 \text{ hidden-to-output} = 200 \text{ weights}$
- **Deep:** $20 + 20 \times 20 \times 3 + 20 = 1240 \text{ weights}$
- **Key difference:** Compositional structure in deep networks

Why Depth Helps: Theoretical Insights

Exponential expressivity:

- Shallow network needs $O(2^n)$ neurons
- Deep network needs $O(n)$ layers
- Each layer can double the number of linear regions

Example: Representing parity function

- Shallow: Needs 2^{n-1} hidden units
- Deep: Needs $O(n)$ units total

Hierarchical composition:

- Layer 1: Edges and gradients
- Layer 2: Simple patterns
- Layer 3: Complex features
- Layer 4: High-level concepts

Approximation Quality vs Network Width

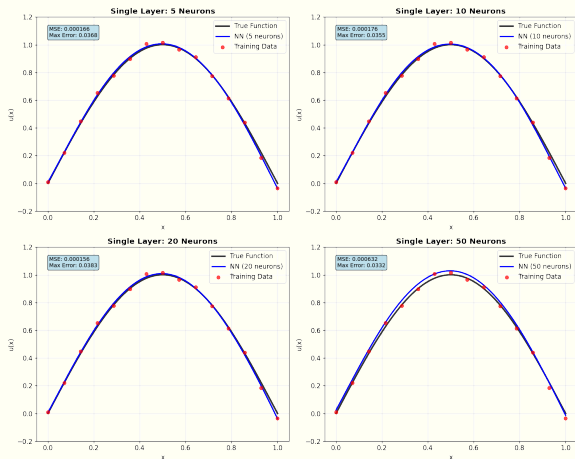


Fig. More neurons improve approximation quality

Convergence Rate of Neural Network Approximation

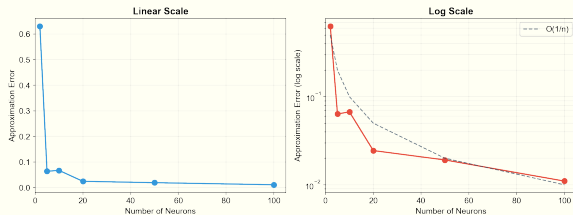


Fig. Error decreases with network width

Empirical observations:

- Error $\sim O(1/\sqrt{N})$ for N neurons
- Diminishing returns after certain width
- Optimal width depends on function complexity

Sobolev Spaces and Derivatives

Definition:

$$H^1([0, 1]) = \{f : \|f\|_{H^1} < \infty\}$$

where

$$\|f\|_{H^1}^2 = \|f\|_{L^2}^2 + \|f'\|_{L^2}^2$$

Why important for PDEs:

- PDEs involve derivatives
- Need to approximate u AND ∇u
- Weak solutions live in Sobolev spaces

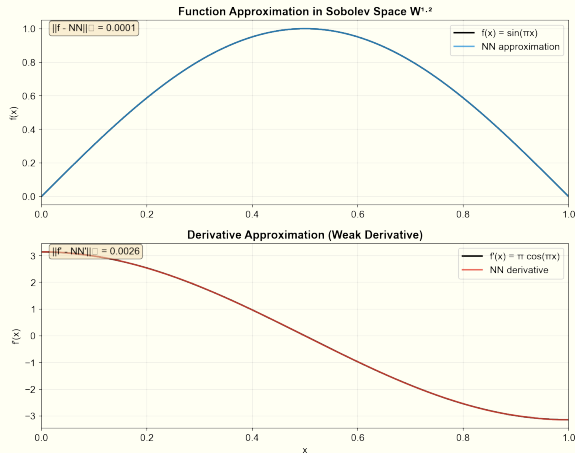


Fig. Neural networks approximate both function and derivative

Physics-Informed Neural Networks (PINNs)

Key idea: Embed physics directly into the loss function

For PDE: $\mathcal{N}[u] = f$ in Ω , $u = g$ on $\partial\Omega$

Loss function:

$$\mathcal{L} = \underbrace{\lambda_1 \|\mathcal{N}[u_\theta] - f\|^2}_{\text{PDE residual}} + \underbrace{\lambda_2 \|u_\theta|_{\partial\Omega} - g\|^2}_{\text{Boundary}} + \underbrace{\lambda_3 \|u_\theta - u_{data}\|^2}_{\text{Data}}$$

Advantages:

- Mesh-free
- Handles high dimensions
- Incorporates physics and data
- Inverse problems

Challenges:

- Balancing loss terms
- Sampling collocation points

Implementation Best Practices

Initialization:

- Xavier/He initialization
- Scale with $1/\sqrt{n_{in}}$
- Avoid symmetry

Architecture:

- Start simple (2-3 layers)
- Width: 50-200 neurons typical
- Add depth for complexity
- Skip connections help

Training:

- Learning rate: $1e-3$ to $1e-4$
- Use Adam optimizer
- Batch normalization

Debugging:

- Check gradient flow
- Monitor activation statistics
- Visualize loss landscape
- Start with simple problems

Common pitfalls:

- Vanishing/exploding gradients
- Dead ReLUs
- Overfitting
- Poor initialization
- Unbalanced data

Tools:

- PyTorch, TensorFlow

Key Takeaways

Mathematical Foundations

- Weierstrass (1885) $\rightarrow$ UAT (1989): continuous functions are learnable
- Neural networks are universal approximators in multiple function spaces
- Density results provide theoretical guarantees

Building Blocks

- Nonlinearity is absolutely essential (XOR problem)
- Hidden layers create new feature spaces
- Activation functions enable complex decision boundaries

Key Takeaways

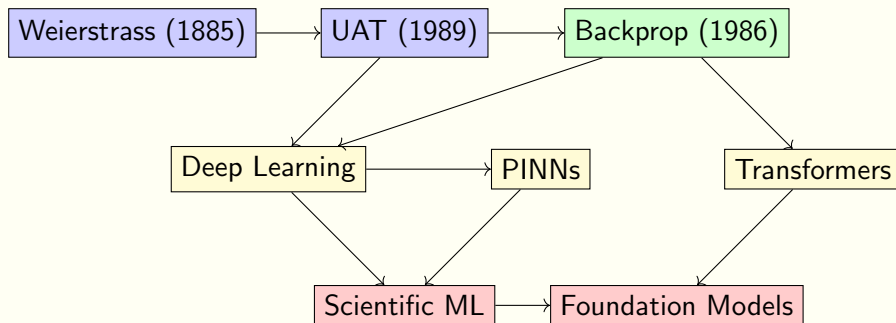
Training and Optimization

- Backpropagation = reverse mode automatic differentiation
- Modern optimizers (Adam) handle complex landscapes
- Regularization prevents overfitting

Architecture Design

- Depth provides exponential expressivity gains
- Width controls approximation quality
- Problem-specific design choices matter

The Path Forward



Current frontiers:

- Neural operators for PDEs
- Geometric deep learning
- Equivariant networks
- Uncertainty quantification

We've journeyed from Weierstrass to modern deep learning

Key insights:

- Neural networks provide a universal, flexible function approximation framework
- Mathematical foundations (UAT) guarantee approximation capabilities
- Nonlinearity + depth = expressivity
- Automatic differentiation enables efficient training
- Applications span from simple classification to solving PDEs

For Scientific ML:

- Mesh-free PDE solvers
- Data-driven discovery
- Multi-physics modeling
- Uncertainty quantification

Questions?

Thank you!

`krishnak@utexas.edu`

`https://github.com/kks32-courses/sciml`